

?Implementation Discussions

1. ActivityInfo authentication

web-monitor will use an ActivityInfo personal API token for authenticated access. The token will not be stored in Git or hard-coded in the application.

For local development, Codex can configure an ignored local environment file and wire the application to read the token from an environment variable. For deployment, the token should be stored in Google Cloud Secret Manager and read by the App Engine service at runtime.

2. ActivityInfo schema and bootstrap

The ActivityInfo database structure should be created through the API as part of a one-time bootstrap process rather than built manually.

The initial schema will center on:

- Monitored Sites: URL, crawl depth, allow/domain restriction, schedule, and active status.
- Crawls: timestamp, status, observed snapshot, crawl metrics, and error/change information.
- Snapshots: one record for each distinct observed state.
- Snapshot Items: child/subform records containing the extracted items used to represent a snapshot.

Snapshot Items should be implemented as an ActivityInfo subform of Snapshot. The bootstrap should create the parent form, add the subform field, define the child schema, and then read the resulting schemas back to verify the relationship.

The bootstrap should also seed roughly three to four monitored sites chosen to demonstrate different rates of change. The initial set should favor structured, crawlable public pages over noisy commercial news homepages. Current candidates include Python.org, Python Insider, IMDb Top 250, and a highly stable IANA page.

Runtime execution should only read and write records; schema creation and schema changes remain explicit bootstrap/migration operations.

3. Query interface and reporting

The monitoring process runs automatically on the backend. The web interface is for

inspecting results and historical state, not for triggering the core diff process.

The initial server-rendered interface can use ordinary HTML forms and date inputs, with little or no JavaScript. Core queries should include:

- state on a selected date;
- what changed on a selected date;
- changes between two dates;
- latest change;
- recent crawl/change history.

Users choose a monitored site rather than a "monitor type." Date-based queries are preferred to raw snapshot selection because the persistence model stores distinct states rather than a full duplicate snapshot on every crawl.

ActivityInfo remains the persistence and aggregate-reporting layer. web-monitor performs crawling, normalization, state resolution, and detailed diffs.

A published ActivityInfo Notebook/report should be created and published through the API where possible. The application can retrieve the published report information and expose a Reports page that embeds the published report, ideally through a simple iframe.

Future direction

AI-assisted semantic extraction is intentionally outside the first implementation. A later enhancement could use AI to identify meaningful content beyond links, suppress cosmetic changes, extract structured information, or summarize detected changes. This belongs in a wishlist/future-work track after the deterministic monitoring pipeline is working.